Team Name : Argmax

Team Leader Name : Sathya T

Problem Statement : Rank the top-100 best-fit candidates from a 100,000-profile pool for the Senior AI Engineer (Founding Team) @ Redrob AI role - contextual + behavioral fit over keyword overlap, CPU-only and offline (<=5 min).

Solution Overview

What is your proposed solution?

- A hybrid, explainable, CPU-only ranking engine: honeypot filter -> JD gated rubric -> semantic + lexical relevance -> cross-encoder rerank -> bounded behavioral multiplier -> fact-grounded reasoning. One command produces the ranked CSV.

What differentiates it from traditional matching?

- Substance over keywords: scores what candidates BUILT (career descriptions); the skills list is excluded from relevance, so keyword-stuffers don't rank.
- Reads profiles, not just text: an impossibility/honeypot filter removes logically-impossible profiles (0 in our top-100 vs the >10% that disqualifies).
- Availability-aware: a bounded behavioral multiplier down-weights perfect-on-paper but unreachable candidates (stale logins, low recruiter response).
- Compliant by design: heavy models run offline (the JD is fixed -> precompute once); the scored step is stdlib, CPU-only, offline, ~190s.

JD Understanding & Candidate Evaluation

Key requirements extracted from the JD

- Must: production embeddings retrieval, vector/hybrid search, strong Python, ranking evaluation (NDCG/MRR/MAP/A-B).
- Profile: 5-9 yrs (ideal 6-8), product company (not pure services/research), shipped an end-to-end ranking/search/recsys system, Pune/Noida or willing to relocate, active on platform.
- Negatives: services-only, CV/speech-without-NLP, title-chasers, recent-LLM-only/framework demos.

Evaluating fit beyond keyword matching

- Career-description evidence of retrieval/ranking work + an engineering-role gate (a 'Marketing Manager' with 9 AI skills scores ~0).
- Skill DEPTH (proficiency x endorsements/duration, corroborated by Redrob assessments), not count.
- Semantic embeddings catch plain-language Tier-5s; 23 Redrob signals weigh real availability.

Ranking Methodology

Retrieve -> score -> rerank

- Recall funnel: fast structured score over all 100k -> keep a top pool.
- Semantic: bge-small-en-v1.5 cosine to a distilled JD query, blended 50/50 with lexical concept matching.
- Rerank: bge-reranker-v2-m3 cross-encoder takes half-authority over the top tier (drives NDCG@10).
- Components (weighted): relevance .30, title/career .28, skill-depth .20, experience .12, education .10.
- Multipliers: penalties (services x0.45, CV/speech x0.6, title-chaser x0.85, location) x behavioral modifier (0.5-1.15).
- Final: score = fit x penalties x behavioral; sort desc, tie-break by candidate_id (matches the validator). Models run offline; rank.py reads cached JSON.

Explainability & Data Validation

How ranking decisions are explained

- Every candidate carries a component breakdown + relevance parts (lexical / semantic / cross-encoder) and a 1-2 sentence reasoning string, shown in the demo UI.

Preventing hallucinations / unsupported justifications

- Reasoning is templated STRICTLY from real profile facts - no LLM in the output path - so a claim cannot reference anything not in the profile. Honest concerns stated; tone matches rank.

Handling inconsistent / suspicious profiles

- Honeypot/impossibility filter (e.g., expert skill with 0 months used; duration > career span). Calibrated to 65 impossible profiles with 0 false positives; 0 reach the top-100.

End-to-End Workflow

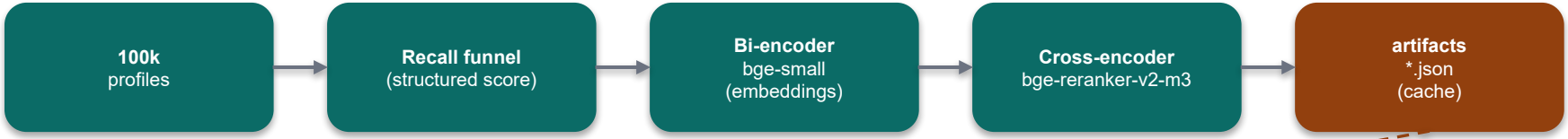
Complete workflow: JD input -> ranked output

- OFFLINE (GPU ok, one-time): embed the candidate pool + cross-encoder rerank -> cache artifacts/*.json.
- SCORED (rank.py, CPU/offline, <=5 min): stream 100k -> honeypot filter -> component scoring x behavioral multiplier -> top-100 heap -> fact-grounded reasoning -> submission.csv.
- Validate: data/validate_submission.py confirms format before upload.

Full workflow diagram: docs/05_approach_and_roadmap.md

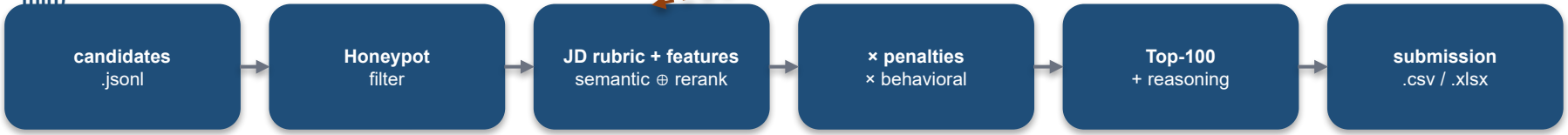
System Architecture

1 · OFFLINE PRECOMPUTE (GPU, one-time)



cached scores → loaded by rank.py

2 · SCORED RANKING — rank.py (CPU · offline · ≤5 min)



Demo sandbox: app/ (FastAPI) + frontend/ (React + Tailwind) → single Docker image (HF Space), reading the same src/. Compliance: GPU / transformers live only in precompute; rank.py is stdlib · CPU · offline.

Results & Performance

Results demonstrating ranking quality

- Validator passes (100 rows, ranks 1-100, scores non-increasing, tie-break by id).
- Honeypots in top-100: 0 - clears the >10% hard disqualifier.
- Quality (proxy, ground truth is hidden): top-100 -> 83 in-band, 97 show ranking work, only 1 junior, 92 available. Top-10 are senior product-company retrieval/ranking/NLP engineers (Meta, Apple, Netflix, Zomato, Paytm, Razorpay...), 5.9-7.9 yrs.

Meeting the runtime / compute constraints

- rank.py: ~190s, CPU-only, offline, <16 GB (the only GPU step is offline precompute).

Method validated by a 3-way query ablation + full-100k negative calibration (docs/09).

Technologies Used

Technologies & why

- Ranker: Python 3.13, stdlib-only - deliberate, guarantees CPU/offline reproducibility at Stage 3.
- Offline precompute: PyTorch + sentence-transformers; bge-small-en-v1.5 (embeddings) + bge-reranker-v2-m3 (cross-encoder) - open-source, no hosted API (Cohere excluded: network).
- Demo: FastAPI; Vite + React + TypeScript + Tailwind; Recharts.
- Tooling: uv (reproducible envs, no pip); just (cross-platform tasks); Docker (HF Space).

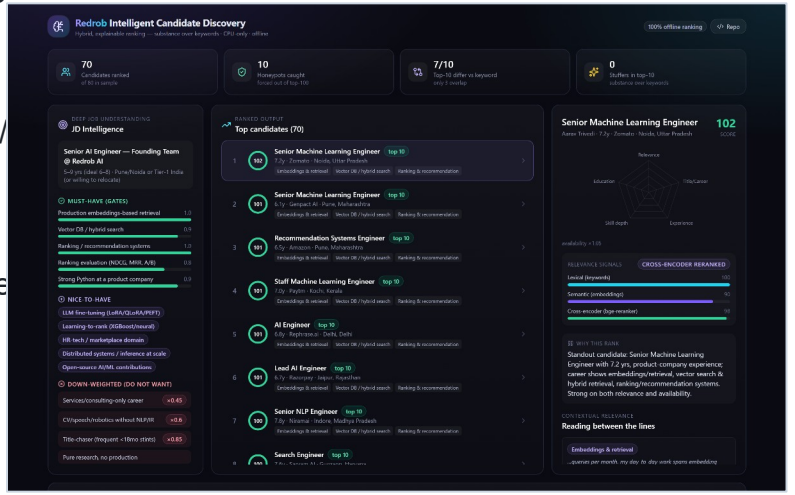
Every choice keeps the scored path inside the compute budget while heavy models run offline.

Submission Assets

Submission assets

- GitHub (public): https://github.com/tsathya98/redrob-candidate-ranker
  - code, docs, single reproduce command: just rank
- Ranked output: submission.xlsx (top-100, candidate_id / rank / score / reasoning).
- This deck (PDF) - approach, methodology, results.
- Docs: README + docs/00-09 + docs/PROJECT_LOG.md (the iteration trail).
  - Run the demo locally: just serve (FastAPI + React) or just docker-run.

Live demo dashboard (just serve)



redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU